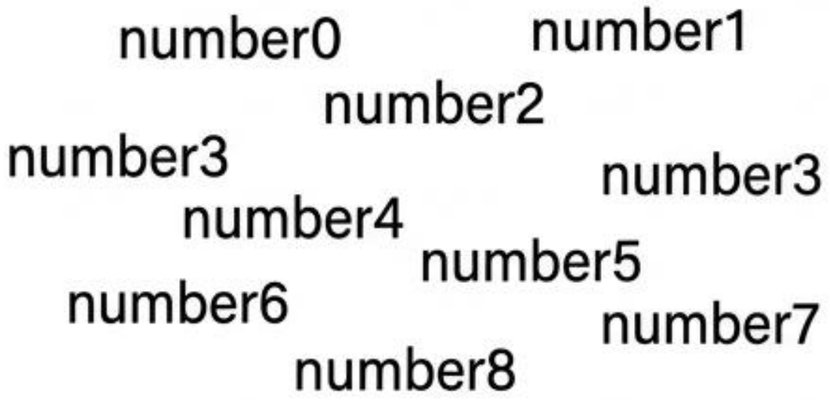
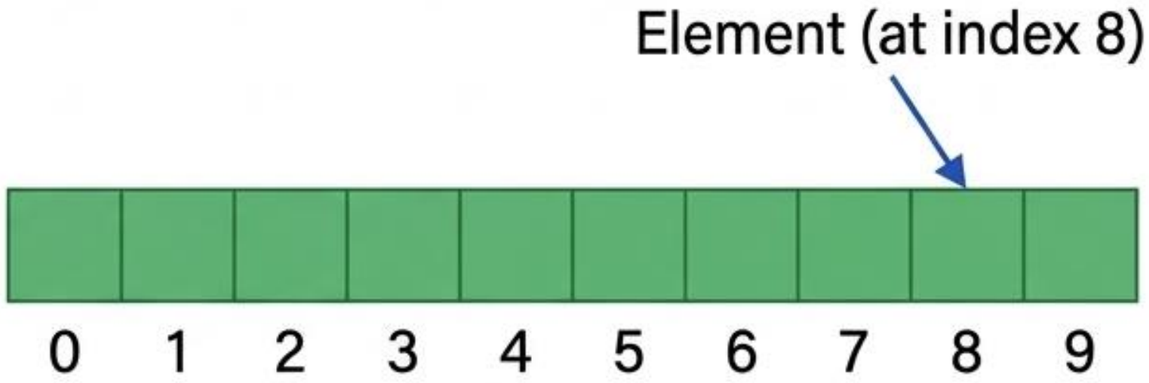


The Array Data Structure

The Foundation: What is an Array?

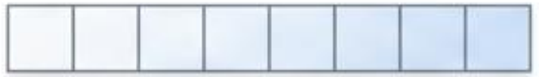
A fixed-size, sequential collection of elements belonging to the same data type. It groups related data under a single variable name for efficiency.





Contiguity: Stored in unbroken, contiguous memory locations.	Base Index: Arrays are zero-indexed (0 is the first element).	Boundary: The last element's index is exactly $Size - 1$.

Array Typology & Declaration Syntax



Single-Dimensional (1D)

Data stored in a linear form.

```
<data_type> <array_name>
[<size>];

int iarr[3] = {2, 3, 4};
char carr[20] = "c4learn";
```

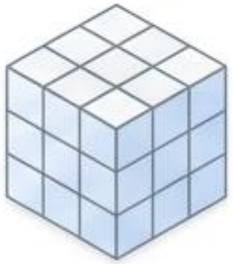


Two-Dimensional (2D / Matrix)

Data stored in tabular format (rows/columns) for mathematical matrices.

```
<data_type> <array_name>
[<row_size>][<column_size>];

int matrix_A[2][3] =
   {{1, 2, 3}, {4, 5, 6}};
```



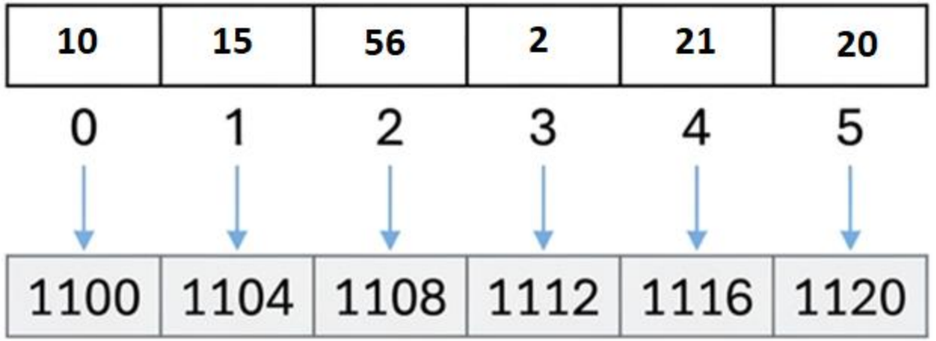
Three-Dimensional (3D)

An “array of 2D arrays”. Accessed using one subscript per dimension.

```
<data_type> <array_name>
[<s1>][<s2>][<s3>] = {values};
```

The Physical Reality: Storing 1D Arrays

Because arrays are contiguous and homogeneous (elements are the same byte size), the CPU instantly calculates any element's exact memory location without scanning.



$$loc(A[i]) = \alpha + (i \times w)$$

$A[0 \ldots x-1]$: An array of size x.

α **(Base Address):** Starting address of the first element (index 0).

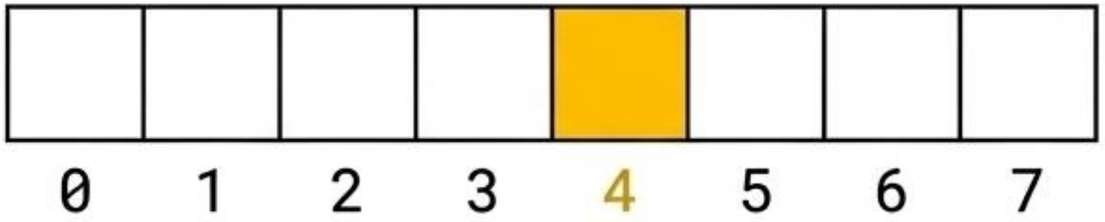
i: Target element index.

w **(Word Size):** Memory space required per element in bytes.

Calculation Lab: 1D Addressing

Calculate the memory address of element $A[4]$ in an array $A[0...7]$ starting at base address 15, where each element requires 8 bytes.

Visual Representation



Mathematical Breakdown

Given Variables: $\alpha = 15$, $i = 4$, $w = 8$

Applied Formula: $loc(A[i]) = \alpha + (i \times w)$

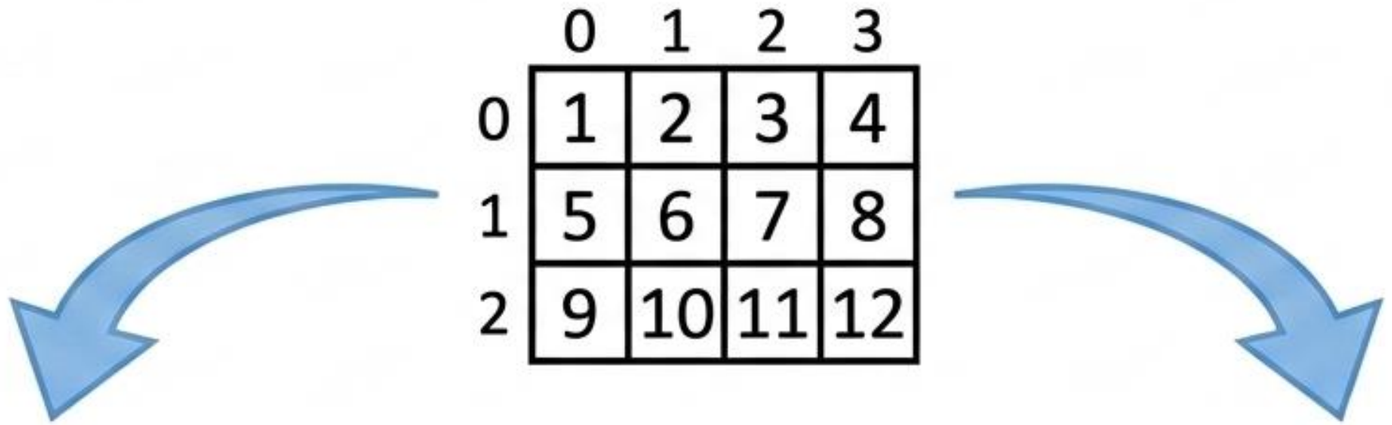
Substitution: $loc(A[4]) = 15 + (4 \times 8)$

Execution: $loc(A[4]) = 15 + 32$

$$loc(A[4]) = 47$$

Flattening the Matrix: 2D Storage Architecture

Physical RAM is strictly one-dimensional. To store a 2D matrix, it must be flattened into a linear sequence using one of two arrangements:



Row-Major Arrangement

1	2	3	4	5	6	7	8	9	10	11	12

Row 0 Row 1 Row 2

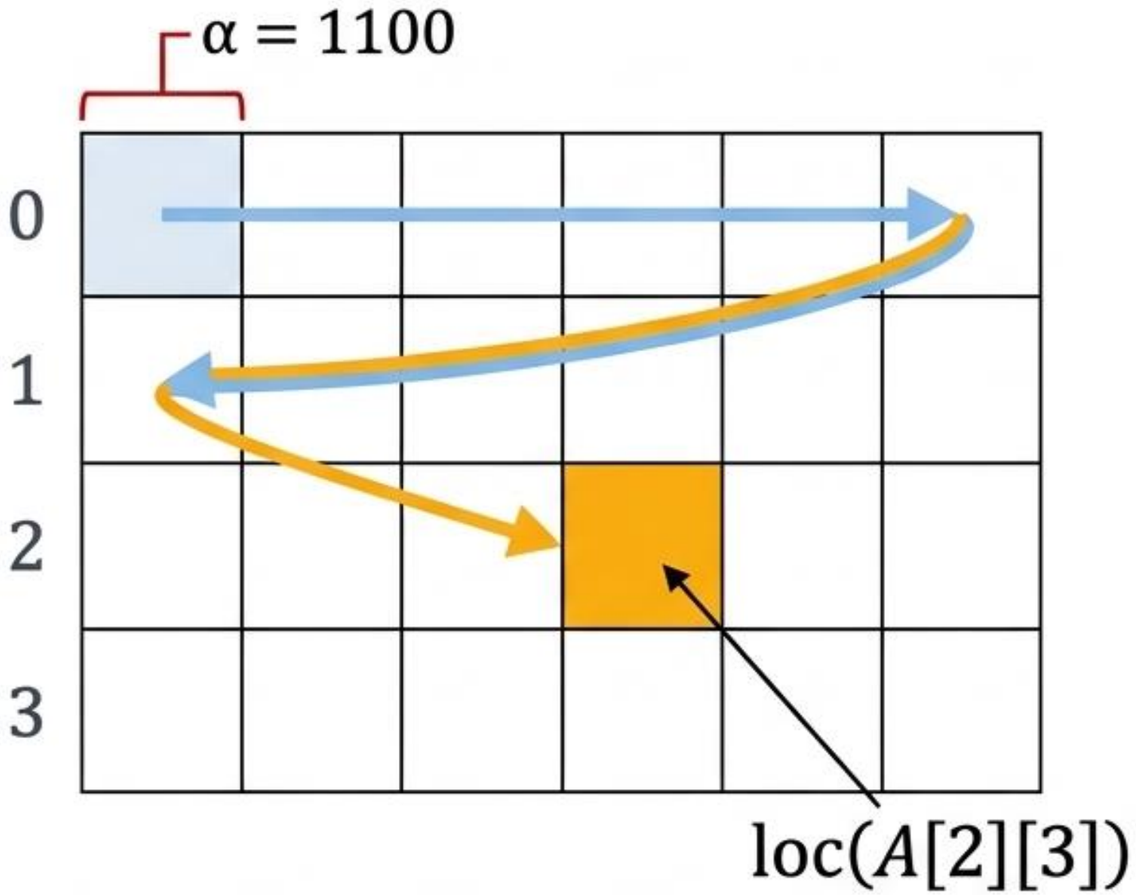
Note: This is the default arrangement evaluated in our mathematical models.

Column-Major Arrangement

1	5	9	2	6	10	3	7	11	4	8	12

Column 0 Column 1 Column 2 Column 3

Calculating 2D Memory Addresses (Row-Major)



$$\text{loc}(A[i,j]) = \alpha + [(i \times y) + j] \times w$$

Boundary Context: Array $A[0...x-1][0...y-1]$.
y = total columns.

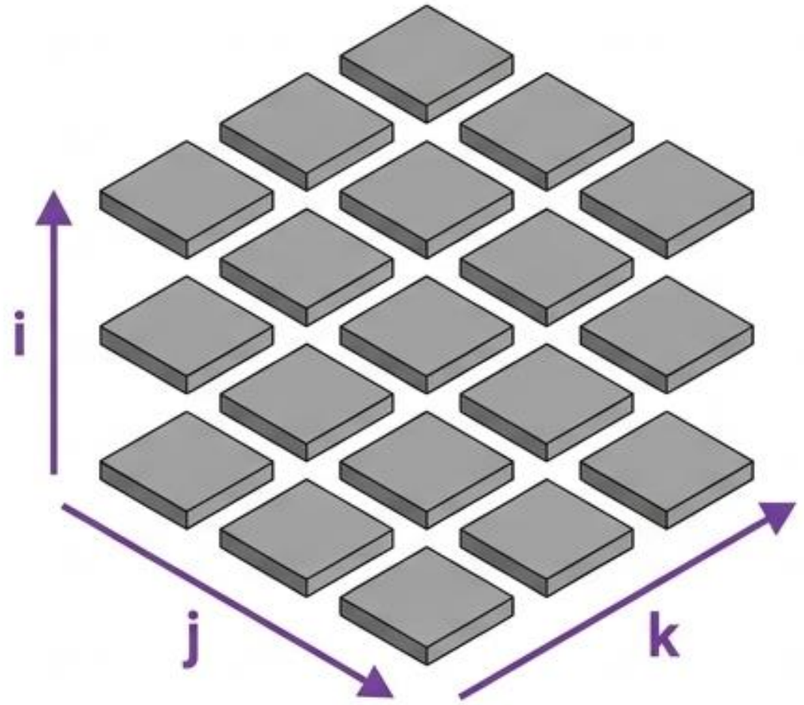
Calculation Lab
Find $A[2][3]$ where total columns $y = 6$.
Base address $\alpha = 1100$, Word size $w = 4$ bytes.

$$\text{Step 1} = 1100 + [(2 \times 6) + 3] \times 4$$
$$\text{Step 2} = 1100 + [12 + 3] \times 4$$
$$\text{Step 3} = 1100 + [15] \times 4$$
$$\text{Step 4} = 1100 + 60$$

1160

Exploring the Third Dimension

A 3D array is structurally stored layer by layer, then row by row. We must calculate the offset for skipped layers, skipped rows in the current layer, and skipped columns in the current row.



$$loc(A[i, j, k]) = \alpha + [(i \times Y \times Z) + (j \times Z) + k] \times w$$

A[0...X-1][0...Y-1][0...Z-1]
Y = Total rows per layer
Z = Total columns per row

Calculation Lab: 3D Addressing

Array B[0...4][0...3][0...5]. The base address is 2000. The element size is 2 bytes. Calculate the exact memory address of **B[3][2][4]**.

Variable Extraction

Total rows $Y = 4$

Total columns $Z = 6$

Base address $\alpha = 2000$

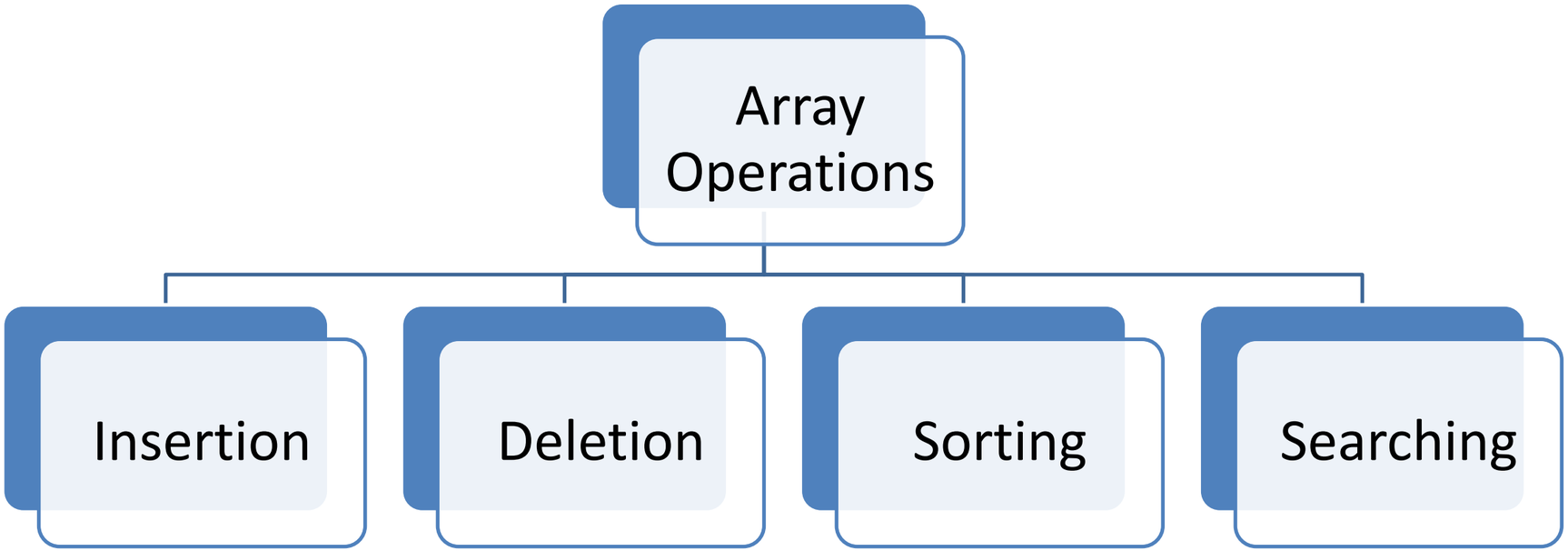
Word size $w = 2$

The Breakdown

$$\begin{aligned} \text{loc}(B[3,2,4]) &= 2000 + [(3 \times 4 \times 6) + (2 \times 6) + 4] \times 2 \\ &= 2000 + [(72) + (12) + 4] \times 2 \\ &= 2000 + [88] \times 2 \\ &= 2000 + 176 \end{aligned}$$

Result: 2176

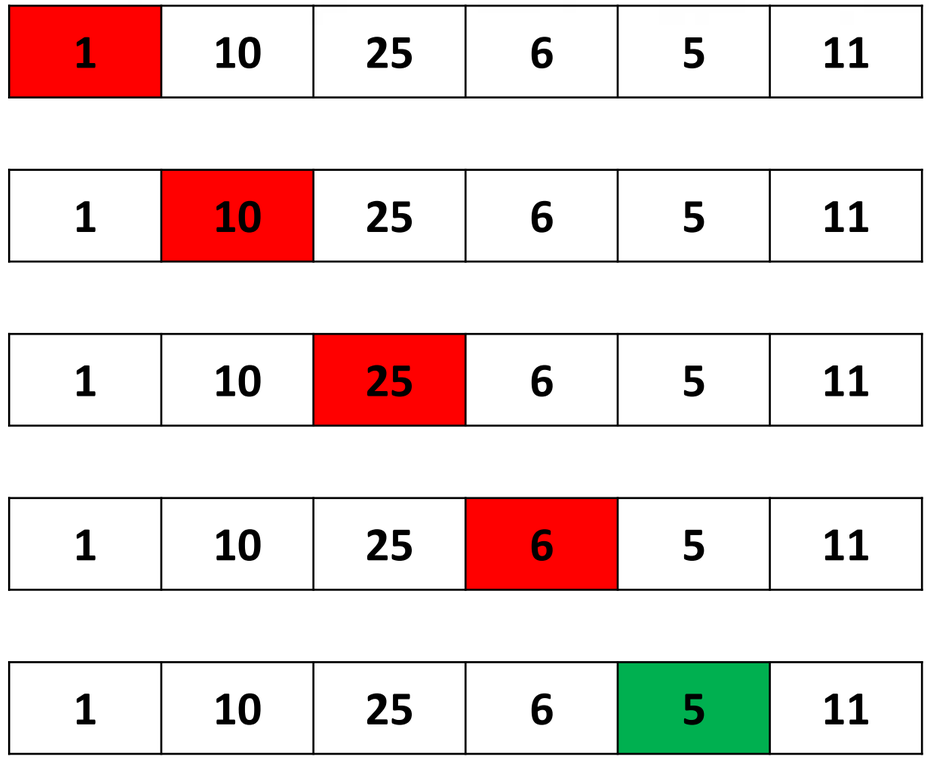
Manipulating the Data: Operations Landscape



Linear (Sequential) Search

Concept: Sequentially compares the target key with every single element until a match is found. Returns index upon success, or -1 if exhausted.

```
Seqsearch(A[], key) {
    for(i = 0; i < length(A); i++)
        if(A[i] == key) return i;
    return -1;
}
```

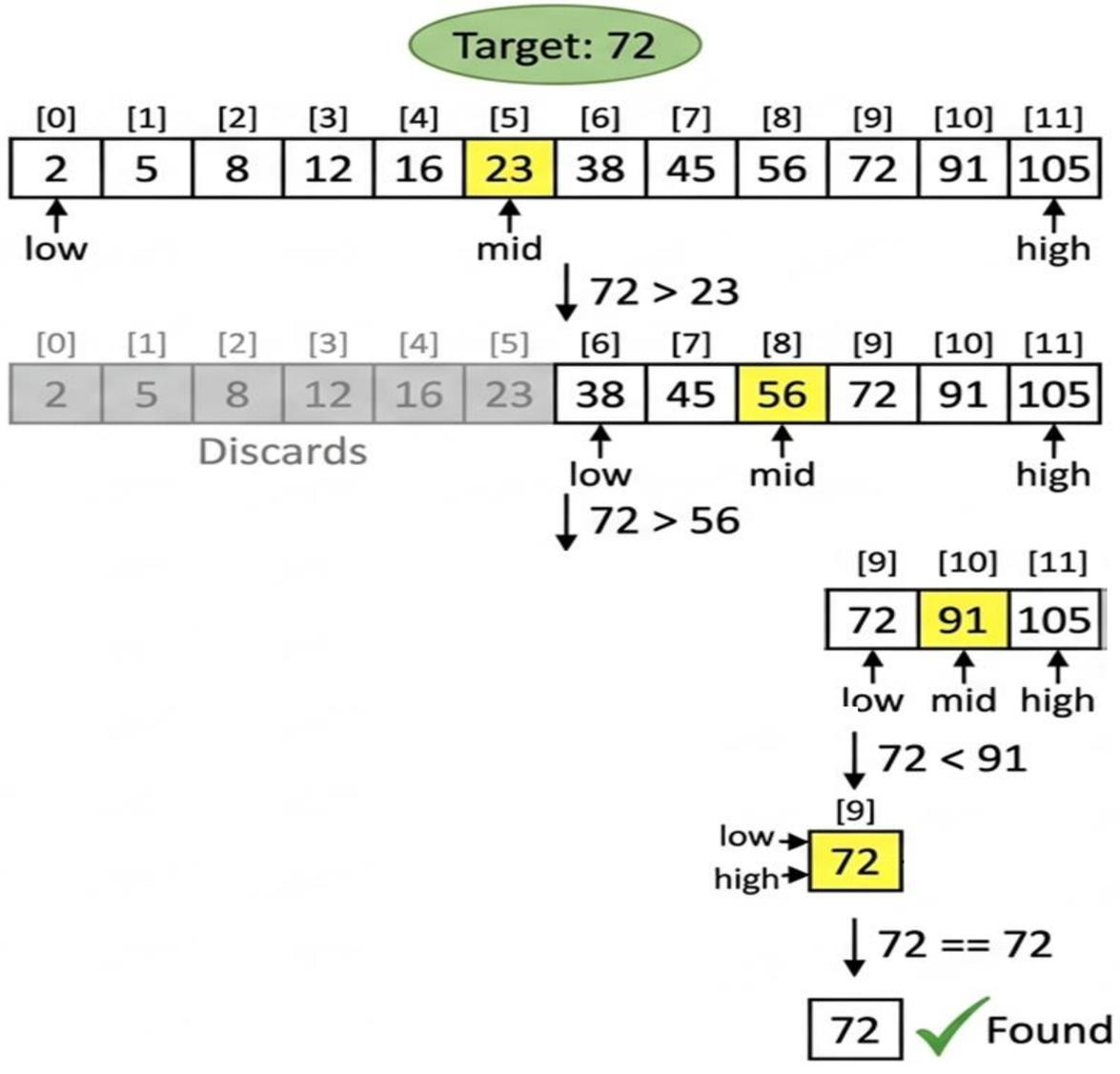


Sorted Data: Binary Search Logic

Mechanism:

Compares key to `mid` element. Discards the irrelevant half, adjusting `low` or `high` pointers.

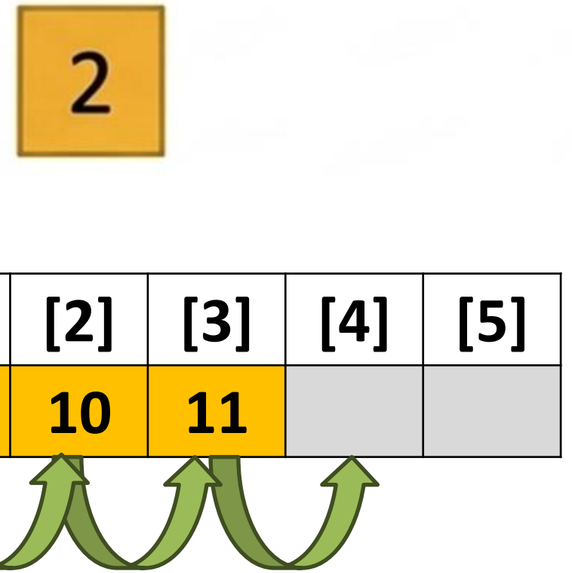
```
while(low <= high) {
    mid = (low + high) / 2;
    if(key === A[mid]) return mid;
    else if(key < A[mid]) high = mid - 1;
    else low = mid + 1;
}
return -1;
```



Mutating Sorted Arrays: Insertion

1. Find target rank position k to maintain sorted order.
2. Shift all elements from the end down to k one unit right.
3. Insert x into vacant A[k].

```
for (i = m - 1; i >= k; i--) {
  A[i + 1] = A[i]; // Shift Right
}
A[k] = x; // Insert
```



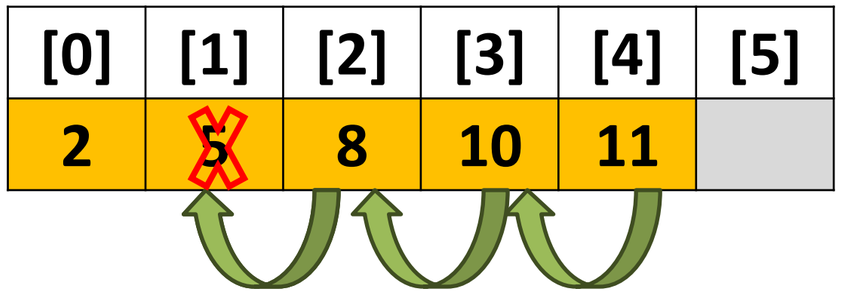
[0]	[1]	[2]	[3]	[4]	[5]
2	5	8	10	11	

Mutating Sorted Arrays: Deletion

1. Locate index k of target.
2. Extract/Save $A[k]$ if required.
3. Shift all elements from $k + 1$ to array's end one unit left to close memory gap.

```
Delete(A[], k) {
  x = A[k];
  for (i = k; i < n; i++) {
    A[i] = A[i + 1]; // Shift Left
  }
  return x;
}
```

[0]	[1]	[2]	[3]	[4]	[5]
2	~~5~~	8	10	11	



[0]	[1]	[2]	[3]	[4]	[5]
2	8	10	11		